PotatoSort AI

Cloud, Data Flow & Storage — Complete Technical Guide

Raspberry Pi 5 YOLOv8 + OpenCV Flask + SQLite Offline-First Architecture

Part 1	Cloud ব্যবহার করা কি Mandatory?
Part 2	Data Flow Logic — Python vs Frontend (JavaScript)
Part 3	Real-Time vs Session-End Sync — সুবিধা-অসুবিধা বিশ্লেষণ
Part 4	Local Storage Options — Database ছাড়া Data Save করার পদ্ধতি
Part 5	SQLite Database Setup, Testing & Verification

Part 1 — Cloud ব্যবহার করা কি Mandatory (বাধ্যতামূলক)?

✓ **সংক্ষিপ্ত উত্তর:** না, Cloud ব্যবহার করা একেবারেই Mandatory বা বাধ্যতামূলক নয়। এটি সম্পূর্ণ আপনার প্রজেক্টের প্রয়োজনের ওপর নির্ভর করে।

Cloud ব্যবহার করবেন কি করবেন না, তা নির্ভর করছে আপনার প্রজেক্টটি **কোথায়** চলবে এবং **কাকে** সেবা দেবে। নিচে দুটি পরিস্থিতি বিশ্লেষণ করা হলো:

কখন Cloud প্রয়োজন নেই — Local / Offline System

আপনার সর্টিং মেশিনটি যদি এমন জায়গায় চলে যেখানে Internet নেই বা দুর্বল — যেমন গ্রামের ক্ষেত বা প্রত্যন্ত এলাকা — তাহলে Cloud-এর কোনো প্রয়োজনই নেই। কারণগুলো নিচে বিস্তারিত দেওয়া হলো:

- Raspberry Pi 5 নিজেই একটি সম্পূর্ণ Mini-Computer। এটি নিজের ভেতরেই SQLite বা MySQL Database Run করতে পারে এবং নিজস্ব Local Web Server (Flask/FastAPI) তৈরি করতে পারে। আলাদা কোনো Server বা Cloud Service কেনার দরকার নেই।
- মোবাইলের Wi-Fi দিয়ে সরাসরি Pi 5-এর Hotspot-এর সাথে কানেক্ট করেই **স্থানীয়ভাবে (Locally)** ওয়েবসাইট বা Dashboard দেখতে পারবেন — ইন্টারনেট ছাড়াই।
- Pi 5-এর IP Address (যেমন: **192.168.4.1**) Browser-এ টাইপ করলেই আপনার Dashboard খুলে যাবে।
- Internet বা Cloud ছাড়াও প্রজেক্ট **১০০%** কাজ করবে — সর্টিং, ডেটা সেভ, রিপোর্ট, সবকিছু।

☁ কখন Cloud ব্যবহার করবেন — Optional Remote Access

Cloud শুধুমাত্র তখনই দরকার যখন আপনি দূরে থেকেও মেশিনের Data দেখতে বা পরিচালনা করতে চান:

- দূর থেকে (Remote Monitoring) অনলাইনের মাধ্যমে Machine-এর Sorting Live দেখতে চাইলে।
- পৃথিবীর যেকোনো প্রান্ত থেকে Data Access করতে বা দীর্ঘমেয়াদে Cloud-এ Data Backup রাখতে চাইলে।
- একাধিক মেশিন বা একাধিক খামারের Data এক জায়গায় একত্রিত করে দেখতে চাইলে।
    - Arduino Cloud — IoT প্রজেক্টের জন্য সহজ, Drag-and-Drop Dashboard তৈরির সুবিধা।
    - Firebase (Google) — Real-Time Database সুবিধা আছে, Mobile App-এর সাথে সংযোগ সহজ।
    - Cloud MySQL / PlanetScale — ডেটা বড় হলে বা SQL Query করতে হলে উপযুক্ত।

📊 Local vs Cloud — পার্থক্য একনজরে

বিষয়	Local System (Pi 5)	Cloud System
Internet দরকার?	না — Hotspot-ই যথেষ্ট	হ্যাঁ — সবসময় লাগবে
Remote Access	নেই (LAN-এর মধ্যে সীমিত)	যেকোনো জায়গা থেকে
খরচ	বিনামূল্যে — Pi 5 নিজেই সব করে	মাসিক Subscription ফি লাগতে পারে
Setup জটিলতা	সহজ — শুধু Pi 5 Config করলেই হয়	বেশি — Cloud Account, API Key লাগে
গ্রাম/ক্ষেতে উপযুক্ততা	✓ **সম্পূর্ণ উপযুক্ত**	✗ Internet ছাড়া কাজ করবে না

> 💡 **পরামর্শ:** CSE 4326 Lab Project-এর জন্য Local System-ই যথেষ্ট এবং সর্বোত্তম। Cloud যোগ করা একটি Optional Extension হিসেবে পরে বিবেচনা করা যায়।

Part 2 — Data Flow Logic (Python vs Frontend JavaScript)

> ✕ **ভুল ধারণা:** UI-এর JavaScript দিয়ে Count করে Database-এ পাঠানো যাবে না — এবং পাঠানো উচিতও নয়।

এর পেছনে ২টি মূল টেকনিক্যাল কারণ রয়েছে যা নিচে বিস্তারিত ব্যাখ্যা করা হলো:

🧠 কারণ ১ — AI Processing Pi 5-এ হয়, UI-তে নয়

YOLOv8 AI Model এবং Camera সরাসরি Raspberry Pi 5-এর **Python Code-এ Backend-এ** Run করে। আলু কখন শনাক্ত হলো এবং Actuator Gate (Servo/Stepper Motor) কখন Fire করবে — এই সম্পূর্ণ সিদ্ধান্ত পাইথন নেয়।

UI (JavaScript) কেবল Browser-এ Screen-এ ছবি বা সংখ্যা দেখায় — সে GPIO Pin, Camera বা Physical Gate নিয়ন্ত্রণ করে না এবং করতেও পারে না।

সহজ কথায়: **Camera → Python → Gate (Physical Action)** — এই Chain-এর কোথাও JavaScript নেই।

🔒 কারণ ২ — Data Security ও Miss-Counting সমস্যা

যদি JavaScript দিয়ে Count করে Database-এ পাঠানো হয়, তাহলে নিচের সমস্যাগুলো হবে:

- **Network Delay:** Browser এবং Server-এর মধ্যে Packet Loss বা Delay হলে কিছু Count হারিয়ে যাবে।

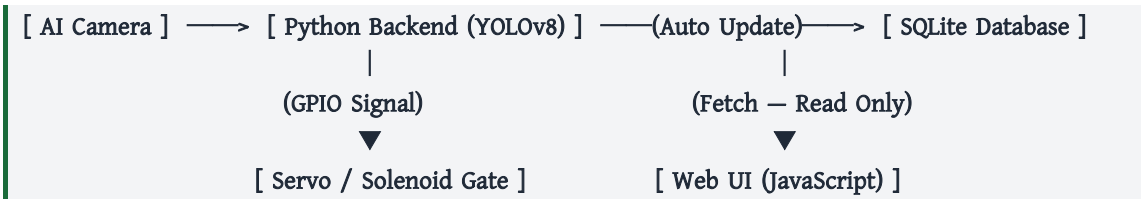
- **Page Refresh:** ব্যবহারকারী Page Refresh করলে JavaScript-এর Memory মুছে যাবে — সব Count শূন্য হয়ে যাবে।
- **Multiple Browser:** একাধিক জায়গা থেকে Dashboard খুললে প্রতিটি Browser আলাদাভাবে Count করবে — হিসাব দ্বিগুণ বা তিনগুণ হয়ে যাবে।
- **Race Condition:** দুটি Detection প্রায় একই সময়ে হলে JavaScript দুটি Request একসাথে পাঠাবে এবং Database-এ শুধু একটি Save হবে।

✓ সঠিক Data Flow Standard — Python → SQLite → UI

সঠিক নিয়ম হলো: **Python (Back-end) → SQLite Database → JavaScript (Front-end)**

স্তর	কে?	কাজ কী?
① Detection	Python	Camera দিয়ে আলু দেখে — YOLOv8 দিয়ে Detect করে Category নির্ধারণ করে।
② Actuator	Python → GPIO	Servo/Stepper Gate Fire করে — আলু সঠিক Bin-এ ফেলে দেয়।
③ DB Write	Python → SQLite	সরাসরি SQLite Database-এ Count ১ বাড়িয়ে Save করে — JS-এর কোনো ভূমিকা নেই।
④ Display	JavaScript	Database থেকে Updated সংখ্যা Read করে (Read-Only) Screen-এ দেখায় — কিছু লেখে না।

Data Flow Diagram



Part 3 — Real-Time vs Session-End Sync

Raspberry Pi 5-এর উচ্চ Processing Power-এ বারবার Database Update করা বা UI-তে Data পাঠানো কোনো চাপই নয়। তবে দুটি পদ্ধতির বাস্তব সুবিধা-অসুবিধা স্পষ্ট জানা দরকার।

✗ Session শেষে একবারে Data পাঠালে যে সমস্যাগুলো হবে

সমস্যা ১ — Real-Time Monitoring সুবিধা নষ্ট হবে

Machine চলাকালীন কোনো নির্দিষ্ট Bin (যেমন: Reject বা Small Bin) পূর্ণ হয়ে গেছে কি না, তা Screen-এ Live দেখা যাবে না। Operator-কে Session শেষ হওয়া পর্যন্ত অপেক্ষা করতে হবে — এই সময়ের মধ্যে Bin উপচে পড়তে পারে।

সমস্যা ২ — Data Loss Risk (সবচেয়ে বড় ঝুঁকি)

> ⚠ **সবচেয়ে বড় ঝুঁকি:** সর্টিং চলাকালীন হঠাৎ বিদ্যুৎ চলে গেলে, System Crash করলে বা কোনো Wire Loose হয়ে Pi 5 বন্ধ হলে — Python-এর RAM-এ জমা পুরো Session-এর হিসাব চিরতরে হারিয়ে যাবে। Database-এ Instant Save না করলে এই ঝুঁকি সবসময় থাকে।

উদাহরণ: ৩ ঘণ্টা ধরে ৪,৫০০টি আলু সর্ট হয়েছে কিন্তু Data Save হয়নি — এমন অবস্থায় বিদ্যুৎ গেলে পুরো Session-এর রিপোর্ট হারিয়ে যাবে।

✓ কেন Real-Time Update কোনো ঝামেলাই নয়?

কারণ ১ — Pi 5 এবং SQLite-এর জন্য এটি অতি সামান্য কাজ

Raspberry Pi 5-এর ARM Cortex-A76 Processor প্রতি সেকেন্ডে হাজার হাজার Database Operation সম্পন্ন করতে পারে। একটি আলু Sort হলে Database-এ **UPDATE daily_summary SET count = count + 1** — এই একটি Query চালাতে মাত্র **০.০০১ সেকেন্ড** সময় লাগে। এটি Pi 5-এর ১%-ও Processing Power ব্যবহার করে না।

কারণ ২ — UI Update করা অত্যন্ত সহজ (Read-Only Polling)

UI (JavaScript) কোনো জটিল কাজ করে না। সে কেবল প্রতি **১ সেকেন্ড** পর পর Python Flask Server-এ একটি ছোট JSON Request পাঠায় এবং Update করা সংখ্যা দেখায়। প্রতিটি JSON Response মাত্র **~200 Bytes** — এতে কোনো Lag বা System-এর ওপর চাপ তৈরি হয় না।

🏆 সেরা পদ্ধতি — In-Memory Counter + Periodic DB Sync

যদি Database-এ ঘনঘন Write করা একেবারেই এড়াতে চান, তাহলে একটি চমৎকার মধ্যবর্তী পদ্ধতি (Best Practice) রয়েছে। এটি ৩টি ধাপে কাজ করে:

ধাপ	কে করে	বিস্তারিত
① RAM Counter	Python	আলু Detect হওয়ার সাথে সাথে Python Memory-তে (RAM Variable-এ) Count যোগ করে: `small_count += 1` — Database-এ কিছু লেখে না, তাই কোনো Disk I/O নেই।
② Live Display	Flask API	Flask API Endpoint (যেমন: `/api/stats`) সরাসরি Python-এর Memory Variable থেকে JSON Return করে। UI এই Live Data দেখায় — Database-এ হাতও দেয় না।
③ Auto Save	Python	প্রতি ৫ মিনিট পর অথবা Stop Button চাপলে Python Memory-র মোট সংখ্যা SQLite Database-এ একবারে Write করে Save করে দেয়।

Optimized Architecture Diagram

```
[ AI Camera ] ──→ [ Python RAM Variables ] ──(Live JSON Stream)──→ [ Web UI Screen ]
                              |                                          |
                  (Auto-Save: every 5 min                  (Read-Only Polling
                   OR on Stop Button press)                 every 1 second)
                              ▼
                  [ SQLite Database (sorter.db) ]
```

> 💡 **ফলাফল:** Database-এ কোনো চাপ নেই — আবার Screen-এ Live Counting-ও দেখা যাচ্ছে! Power Cut হলেও সর্বোচ্চ ৫ মিনিটের Data হারাবে, পুরো Session নয়।

📊 Real-Time vs Periodic vs Session-End — তুলনামূলক বিশ্লেষণ

বৈশিষ্ট্য	Real-Time (প্রতি Detection)	Periodic (৫ মিনিট পর)	Session-End (একবারে)
Live Dashboard	✓✓ সবচেয়ে সঠিক	✓✓ প্রায় Live	✗ দেখা যাবে না
Power Cut-এ Data Loss	✓✓ শূন্য Loss	⚠ সর্বোচ্চ ৫ মিনিট	✗ সম্পূর্ণ Loss
DB-এর ওপর চাপ	⚠ সামান্য বেশি	✓✓ নগণ্য	✓✓ একেবারে নেই

Pi 5-এ উপযুক্ততা	✓ সম্পূর্ণ উপযুক্ত	✓ সেরা Balance	✗ Risk বেশি

Part 4 — Local Storage Options (Database ছাড়া)

হ্যাঁ, Database ছাড়াই Locally Data Save করা সম্পূর্ণ সম্ভব। SQLite বা অন্য কোনো Database Install না করেও Raspberry Pi 5-এ ৩টি উপায়ে Data জমা রাখা যায়:

① JSON ফাইল — সবচেয়ে জনপ্রিয় ও সহজ পদ্ধতি

Python-এর Memory-তে (RAM-এ) চলা হিসাবগুলো Session শেষে বা প্রতি ৫ মিনিট পর পর একটি .json ফাইলে Save করে রাখা যায়। Python-এর Built-in **json** Library ব্যবহার করে Data একটি Dictionary আকারে Text File-এ লেখা হয় — কোনো External Library Install করতে হয় না।

কীভাবে কাজ করে:

- Sorting চলার সময় Python Memory Variable-এ Count রাখে (যেমন: **small_count = 0**)।
- প্রতি ৫ মিনিট পর বা Stop Button চাপলে এই Variables-গুলো একটি Dictionary-তে রাখা হয়।
- সেই Dictionary-টি **json.dump()** দিয়ে **summary.json** ফাইলে লেখা হয়।

ফাইলের নমুনা — summary.json

```
{
  "date": "2026-09-13",
  "total_scanned": 1500,
  "reject": 200,
  "small": 400,
  "medium": 600,
  "large": 300
}
```

Python Code — JSON-এ Save করার পদ্ধতি

```
import json
```

```
from datetime import datetime


def save_to_json(counts):
    data = {
        "date":           datetime.now().strftime("%Y-%m-%d"),
        "total_scanned": counts["total"],
        "reject":         counts["reject"],
        "small":          counts["small"],
        "medium":         counts["medium"],
        "large":          counts["large"],
    }
    with open("summary.json", "w") as f:
        json.dump(data, f, indent=2)
    print("Saved to summary.json")
```

✓ **সুবিধা:** পড়তে এবং Update করতে অত্যন্ত সহজ। UI-তে Data পাঠানোও সহজ — Flask সরাসরি JSON File Return করতে পারে।

i **নিরাপত্তা:** শুধুমাত্র Pi 5-এর Python Code-টিই এই ফাইলে Write করবে — তাই File Corrupt হওয়ার কোনো ঝুঁকি নেই।

② CSV ফাইল — Excel-এ খোলার জন্য সেরা

আলুর Sorting হিসাব সরাসরি Comma-Separated Values ফাইলে (Spreadsheet আকারে) Save করা যায়। এই ফাইল সরাসরি USB পেনড্রাইভে নিয়ে যেকোনো কম্পিউটারে **Microsoft Excel** বা **Google Sheets** দিয়ে খুলে রিপোর্ট দেখা যায়।

ফাইলের নমুনা — daily_report.csv

```
Date,Total,Reject,Small,Medium,Large
2026-09-12,1200,150,300,500,250
2026-09-13,1500,200,400,600,300
```

Python Code — CSV-এ Row যোগ করার পদ্ধতি

```
import csv
from datetime import datetime


def append_to_csv(counts):
    today = datetime.now().strftime("%Y-%m-%d")
    row   = [today, counts["total"], counts["reject"],
```

```
            counts["small"], counts["medium"], counts["large"]]

    with open("daily_report.csv", "a", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(row)
    print("Row appended to CSV")
```

✓ **সুবিধা:** Excel-এ সরাসরি Chart বানানো যায়, Filter করা যায়। পেনড্রাইভে নিয়ে অন্য Computer-এ Report দেখা যায়।

⚠ **সীমাবদ্ধতা:** প্রতিদিন নতুন Row যোগ হতে থাকে — ফাইল বড় হয়। নির্দিষ্ট তারিখের Data খুঁজতে SQL-এর মতো Query করা যায় না।

③ সাধারণ Text ফাইল (.txt) — সবচেয়ে সহজ

খুবই সাধারণ একটি .txt ফাইলে প্রতিদিনের শেষ সংখ্যাটি লিখে রাখা। যখন শুধু একটি সংখ্যা Track করতে হয় এবং Report-এর দরকার নেই, তখনই এটি উপযুক্ত।

ফাইলের নমুনা — last_count.txt

```
2026-09-13 | Total: 1500 | Reject: 200 | Small: 400 | Medium: 600 | Large: 300
```

তিনটি পদ্ধতির তুলনা

পদ্ধতি	সেরা কখন	সীমাবদ্ধতা	Python Library
JSON File	Simple Project, UI-তে সরাসরি Data	Multi-thread Write-এ ঝুঁকি (Pi 5-এ নেই)	**import json**
CSV File	Excel-এ Report দরকার হলে	File বড় হয়, Query করা যায় না	**import csv**
TXT File	Ultra-Simple, শুধু Last Count	History নেই, Report বানানো কঠিন	**open() built-in**
SQLite DB	Long-Term, Query, History দরকার হলে	সামান্য বেশি Setup — তবে সবচেয়ে Reliable	**import sqlite3**

> 💡 **সুপারিশ:** Lab Project-এর জন্য সেরা পছন্দ — SQLite Database। কারণ এতে History Query করা যায়, Multiple Day-এর Report বের করা যায় এবং Python-এর sqlite3 Built-in থাকায় কোনো Install লাগে না।

Part 5 — SQLite Database Setup & Testing

> ℹ **গুরুত্বপূর্ণ:** Raspberry Pi 5-এ কোনো বাড়তি Software Install করতে হবে না। Python-এর নিজস্ব sqlite3 Library ব্যবহার করেই Database তৈরি, Data Insert এবং Read করা সম্ভব।

নিচে ৩টি ধাপে Database তৈরি ও Test করার পুরো পদ্ধতি দেওয়া হলো। এই Code সরাসরি Pi 5-এ চালালেই কাজ করবে।

Step 1 — টেবিল তৈরির Script (Database Initialization)

আপনার **main.py** ফাইলের একদম শুরুতে নিচের Code টি যুক্ত করুন। এই Code-টি প্রজেক্ট ফোল্ডারে **potato_sorter.db** নামের ফাইল তৈরি করবে (যদি আগে না থাকে) এবং তার ভেতর **daily_summary** টেবিল বানাবে:

```
import sqlite3

def init_db():
    # প্রজেক্ট ফোল্ডারে potato_sorter.db ফাইল তৈরি হবে
    conn   = sqlite3.connect('potato_sorter.db')
    cursor = conn.cursor()

    # আজকের দিনের Summary রাখার জন্য Table তৈরি
    cursor.execute('''
        CREATE TABLE IF NOT EXISTS daily_summary (
            date          TEXT    PRIMARY KEY,   -- YYYY-MM-DD
            total_scanned INTEGER DEFAULT 0,
            reject_count  INTEGER DEFAULT 0,
            small_count   INTEGER DEFAULT 0,
            medium_count  INTEGER DEFAULT 0,
            large_count   INTEGER DEFAULT 0
```

```
        )
    ''')

    conn.commit()
    conn.close()
    print("Database initialized successfully!")

# Program চালু হওয়ার সাথে সাথে Database তৈরি হবে
init_db()
```

Column-গুলো কী কাজ করে?

Column নাম	Data Type	বিবরণ
date	TEXT	Primary Key — একই তারিখে দুটি Row হবে না (YYYY-MM-DD Format)
total_scanned	INTEGER	দিনে মোট কতটি আলু Scan হয়েছে
reject_count	INTEGER	Reject / Blight Category-তে পড়েছে কতটি
small_count	INTEGER	Small (<50mm) Category-তে পড়েছে কতটি
medium_count	INTEGER	Medium (50-75mm) Category-তে পড়েছে কতটি
large_count	INTEGER	Large (>75mm) Category-তে পড়েছে কতটি

Step 2 — Test Data Insert করে Database যাচাই

নিচের Python Code টি চালালে Database-এ একটি Test Data Insert হবে। **ON CONFLICT ... DO UPDATE** অংশটি নিশ্চিত করে যে একই তারিখে দুটি Row তৈরি না হয়ে শুধু Count বাড়ে।

```
from datetime import datetime
import sqlite3

def add_test_data():
    today  = datetime.now().strftime('%Y-%m-%d')
    conn   = sqlite3.connect('potato_sorter.db')
    cursor = conn.cursor()

    # Test Data Insert — যদি আজকের তারিখ থাকে, শুধু Count বাড়বে
```

```
    cursor.execute('''
        INSERT INTO daily_summary
            (date, total_scanned, reject_count, small_count, medium_count, large_count)
        VALUES (?, 100, 10, 30, 40, 20)
        ON CONFLICT(date) DO UPDATE SET
            total_scanned = total_scanned + 100,
            reject_count  = reject_count  + 10,
            small_count   = small_count   + 30,
            medium_count  = medium_count  + 40,
            large_count   = large_count   + 20
    ''', (today,))

    conn.commit()
    conn.close()
    print('Test data inserted successfully!')

add_test_data()
```

✓ **ব্যাখ্যা:** ON CONFLICT(date) DO UPDATE — এই SQL Clause-টি UPSERT নামে পরিচিত। এটি বলছে: "যদি এই তারিখের Row আগে থেকেই থাকে, নতুন Row বানিও না — শুধু Count-গুলো বাড়াও।"

Step 3 — Database-এর Data যাচাই করবেন কীভাবে?

Raspberry Pi 5-এর Terminal থেকেই Database-এ প্রবেশ করে Data দেখা ও যাচাই করা যায়। নিচে ধাপে ধাপে দেখানো হলো:

ধাপ ১ — Terminal-এ Database ফাইল Open করুন

```
sqlite3 potato_sorter.db
```

এটি চালালে Terminal-এর Prompt বদলে **sqlite>** হবে — এখন আপনি SQLite-এর ভেতরে আছেন।

ধাপ ২ — সব Data দেখুন

```
SELECT * FROM daily_summary;
```

সফল হলে নিচের মতো আউটপুট দেখাবে:

```
2026-09-13|1500|200|400|600|300
```

ধাপ ৩ — Column নাম সহ সুন্দরভাবে দেখুন

```
.headers on
.mode column
SELECT * FROM daily_summary;
```

এখন Column নাম সহ সুন্দর Table আকারে দেখাবে।

ধাপ ৪ — নির্দিষ্ট তারিখের Data দেখুন

```
SELECT * FROM daily_summary WHERE date = '2026-09-13';
```

ধাপ ৫ — SQLite থেকে বের হোন

```
.exit
```

> ✓ **সফল হলে দেখবেন:** Terminal-এ date | total_scanned | reject_count | small_count | medium_count | large_count Column সহ Data Row প্রদর্শিত হবে।

✎ Bonus — Flask API দিয়ে UI-তে DB Data পাঠানো

নিচের Flask Endpoint টি যোগ করলে Dashboard JavaScript সরাসরি এই URL থেকে JSON Data নিয়ে Screen-এ দেখাতে পারবে:

```
from flask import Flask, jsonify
import sqlite3

app = Flask(__name__)

@app.route('/api/today')
def get_today():
    conn   = sqlite3.connect('potato_sorter.db')
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM daily_summary WHERE date = date('now')"    )
    row    = cursor.fetchone()
    conn.close()

    if row:
        return jsonify({
```

```
            "date":           row[0],
            "total_scanned": row[1],
            "reject":         row[2],
            "small":          row[3],
            "medium":         row[4],
            "large":          row[5],
        })
    return jsonify({"message": "No data for today yet."})
```

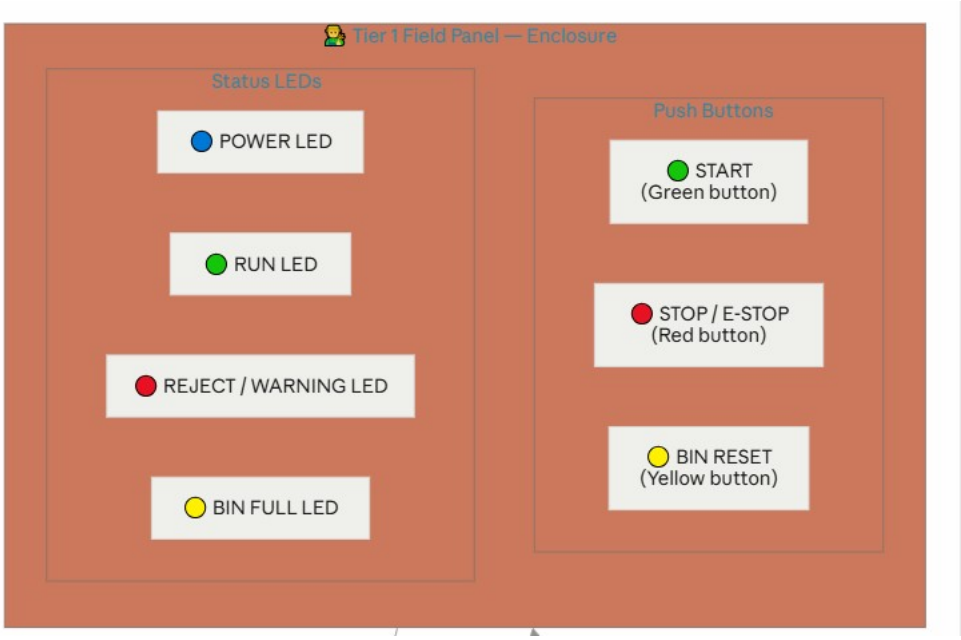
> 💡 **ব্যবহার:** Browser বা JavaScript থেকে http://192.168.4.1:5000/api/today এ GET Request পাঠালে JSON আকারে আজকের সব Count পাবেন।

PotatoSort AI — Physical UI Block Diagrams (Tier 1 & Tier 2)

Legend: ✓ = code-এ implemented · ⚠ = আংশিক / metadata-only · ✗ = wired কিন্তু কোনো functional logic নেই

Tier 1 — Farmer Profile Panel (No Screen)

টার্গেট ইউজার: গ্রামীণ কৃষক — plug-and-play, ultra-simple।

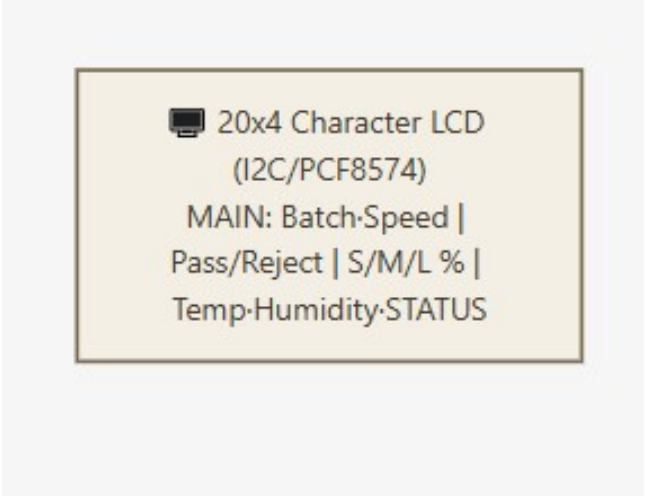


কি কি থাকবে — Component List

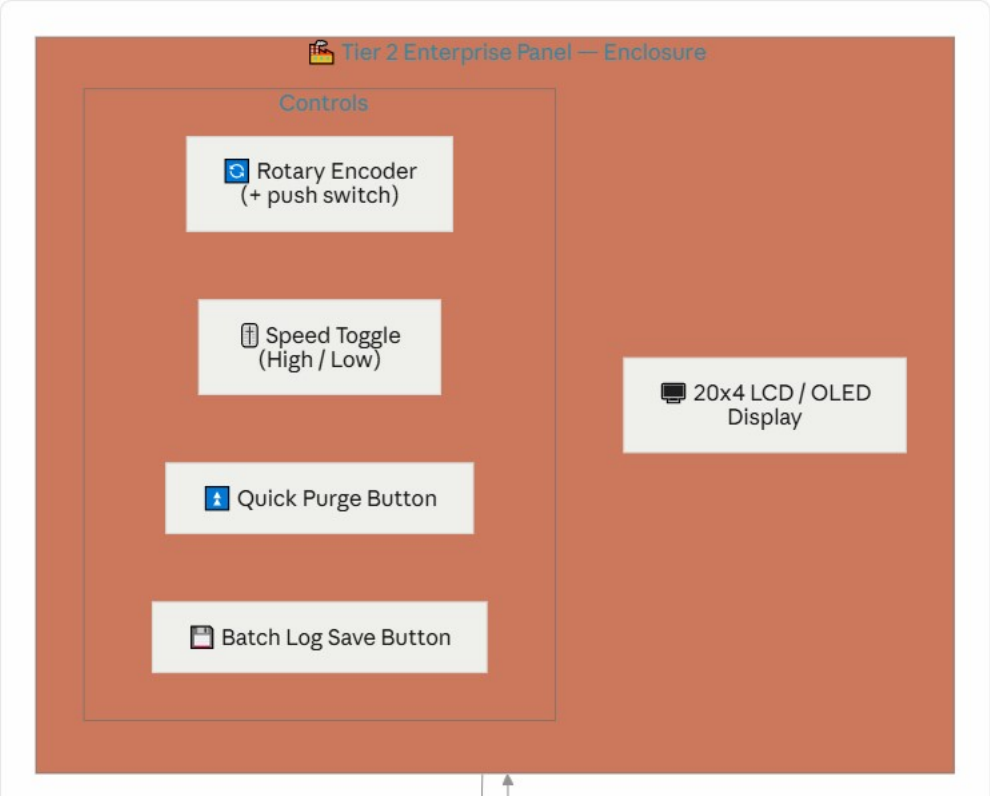
#	Component	রঙ	কাজ	কোডে অবস্থা
1	START button	সবুজ	Conveyor belt + camera + sorting logic চালু করা	✓ PIN_BTN_START → on_start
2	STOP / E-Stop button	লাল	মুহূর্তেই পুরো লাইন বন্ধ	✓ PIN_BTN_STOP → on_stop
3	BIN RESET button	হলুদ	ঝুড়ি খালি করার পর শুধু count reset (belt চালু থাকা অবস্থায়)	⚠ বর্তমানে পুরো সিস্টেম IDLE করে দেয় — belt-running অবস্থায় শুধু bin-count reset করার আলাদা path নেই
4	Power LED	নীল	পাওয়ার কানেকশন active কিনা	⚠ কোডে আছে কিন্তু README wiring-এ রঙ Green লেখা — mismatch
5	Run LED	সবুজ	সর্টিং চলছে কিনা (blink)	⚠ কোডে আছে কিন্তু README wiring-এ রঙ Blue লেখা — mismatch
6	Reject/Warning LED	লাল	আলু আটকে গেলে বা reject bin ভর্তি হলে alert	✗ বর্তমানে শুধু CPU%/temperature threshold-এ জ্বলে; potato-jam detection নেই
7	Bin Full LED	হলুদ	নির্দিষ্ট bin ভরে গেলে alert	✗ GPIO pin ও state field আছে, কিন্তু কোনো bin-full sensor/logic কখনো এটাকে True করে না

Tier 2 — Enterprise Operator Panel (Cold Storage / Trader)

টার্গেট ইউজার: কোল্ড স্টোরেজ অপারেটর ও পাইকারি ব্যবসায়ী — batch monitoring, environment tracking, দ্রুত truck loading।



Optional for tier 2 onley:



কি কি থাকবে — Component List

#	Component	কাজ	কোডে অবস্থা
1	20x4 LCD / OLED Display	Batch নম্বর, timer, Pass/Reject count, 8 বিনের %/weight, Temp & Humidity	⚠ শুধু Batch ID, L/M/S/R raw count, reject%, CPU temp দেখায়। Timer, %/weight, humidity — কোনোটাই নেই
2	Rotary	মেনু navigation, size-	✕ শুধু CW/CCW direction log করে; menu

#	Component	কাজ	কোডে অবস্থা
	Encoder	threshold পরিবর্তন, calibration	বা runtime threshold change implement করা হয়নি
3	Speed Toggle Switch	High/Low speed মোড — দ্রুত loading-এর জন্য belt speed বদলানো	⚠ state-এ HIGH/LOW সেভ হয় ও DB-তে log হয়, কিন্তু belt motor-কে actually control করার কোনো কোড নেই
4	Quick Purge Button	Belt না থামিয়ে সর্বোচ্চ speed-এ পুরো conveyor খালি করা	✗ request_purge() flag সেট হয়, log হয় — কিন্তু flag-টা কোথাও read/consume হয় না, তাই আসলে belt speed বাড়ে না
5	Batch Log Save Button	Batch শেষে count SQLite-এ স্থায়ীভাবে সেভ	✓ on_save → _save_batch_to_db() — সম্প্রতি idempotent করে fix করা হয়েছে

সারসংক্ষেপ — যা যোগ করা দরকার

1. **Tier 1:** LED রঙ ঠিক করা (README ↔ spec একরকম করা), Bin-Reset কে belt-running অবস্থায় শুধু count-reset করার আলাদা command বানানো, Bin-Full sensor input যোগ করা।
2. **Tier 2:** Rotary encoder দিয়ে actual menu/threshold-calibration লজিক, Purge flag consume করে belt-speed বাড়ানোর কোড, environmental temp/humidity sensor (DHT22/SHT31 জাতীয়) integrate করা, OLED-এ timer + %/weight যোগ করা।